VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Parallel Computing

EXERCISES

Instructor(s): Le Thanh Sach

Group Members:

Tran Ngoc Dat - 2570388

Nguyen Dinh Quang Khai - 2570231

HO CHI MINH, October, 2026

Table Contents

Table Contents **3**
1. Dataset **4**
1.1 CIFAR-10 Dataset Overview 4
2. Exercises **5**
2.1 E1 5
2.1.1 SoftMax Classifier 5
2.1.2 Multilayer Perceptron Classifier 5
2.1.3 Convolutional Neural Network Classifier 5
2.1.3.1 Introduction 5
2.1.3.2 Overview 5
2.1.3.3 Discussion And Experimental Analysis 6
2.2 E2 7
2.3 E3 7
3. References **7**

1. Dataset

1.1 CIFAR-10 Dataset Overview

The **CIFAR-10** dataset is a standard computer vision dataset. It consists of 60,000 RGB (3 Channels) images, 32x32 pixels, containing one of 10 object classes, with 6,000 images per class.
The **CIFAR-10 dataset** collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton.
There are 50,000 training images, and 10,000 test images

airplane										
automobile										
bird										
cat										
deer										
dog										
frog										
horse										
ship										
truck										

2. Exercises

2.1 E1

2.1.1 SoftMax Classifier

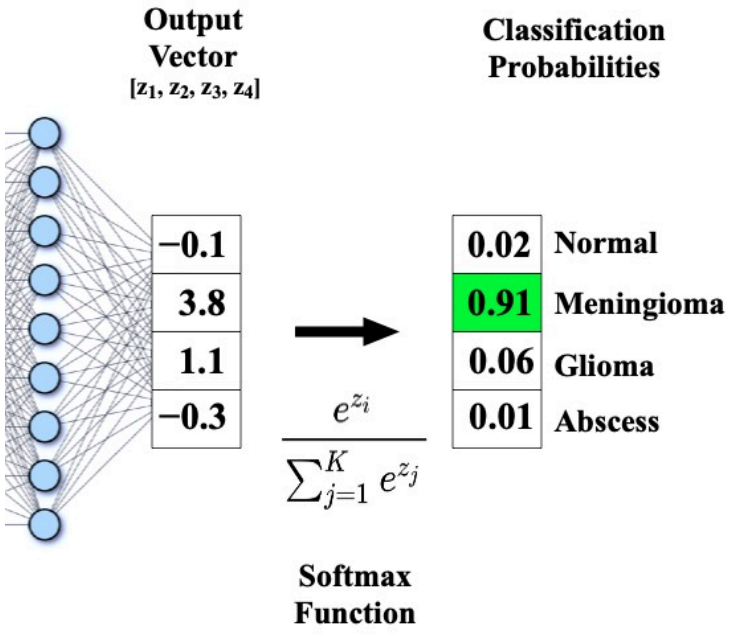
2.1.3.1 Introduction

A **softmax classifier** (also known as multinomial logistic regression or maximum entropy classifier) is a machine learning model used for multi-class classification

How does the Softmax Classifier Work?

- In a Softmax Classifier, the neural network outputs a set of raw scores for each class. These raw scores, also called logits, are then passed through the Softmax function, converting them into probabilities. The class with the highest probability is chosen as the model's prediction.

Softmax Function: $p_i = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$



2.1.3.3 Discussion And Experimental Analysis

2.1.3.3.1 Softmax Configuration

Layer	Input	Output

Flatten	tensor([3,32,32))	vector([x1, x2,...,x3072]); 3072 pixels
Linear: Linear(3 * 32 * 32, num_classes)	vector([x1, x2,...,x3072]); 3072 pixels	a vector of 10 logits, with each logit corresponding to a class

2.1.3.3.2 Analysis

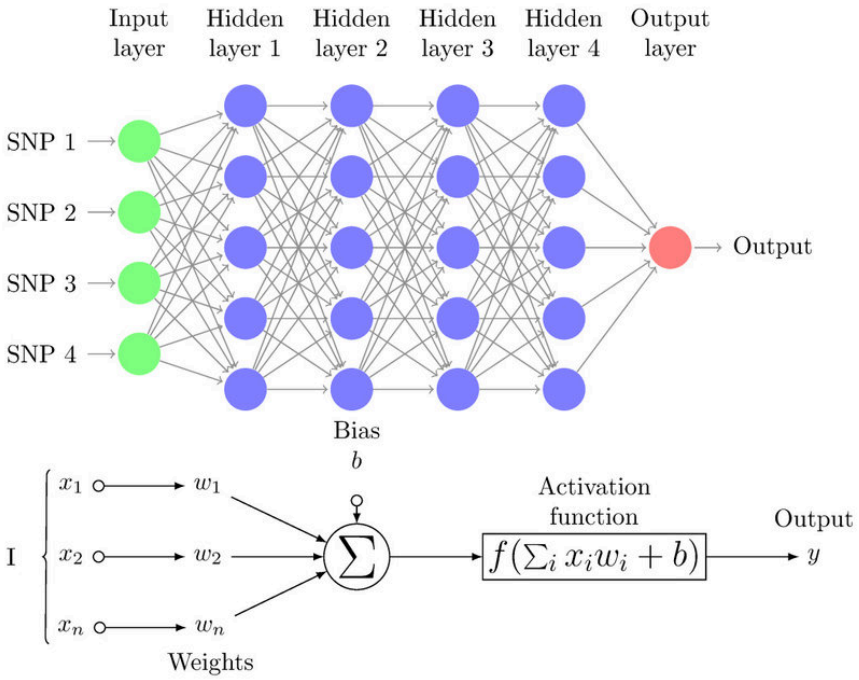
2.1.2 Multilayer Perceptron Classifier

2.1.2.1 Introduction

A Multi-Layer Perceptron is a type of neural network that consists of multiple layers of nodes, or "neurons." These layers include -

- Input Layer: This is where the data enters the network. Each neuron in this layer represents one feature of the input data.
- Hidden Layers: These are intermediate layers between the input and output layers. An MLP can have one or more hidden layers, which allow the network to learn complex patterns.
- Output Layer: This layer produces the final result, such as a class label or a numeric value, depending on the task at hand.

Each neuron in a layer is connected to every neuron in the next layer, and these connections have weights that are adjusted during training to improve the network’s performance.



2.1.2.2 Discussion And Experimental Analysis

2.1.3 Convolutional Neural Network Classifier

2.1.3.1 Introduction

A Convolutional Neural Network (CNN) is a specialized type of deep learning algorithm mainly designed for tasks that necessitate object recognition, including image classification, detection, and segmentation. CNN is inspired by the human visual system, designed to process image data by capturing spatial relationships between pixels.

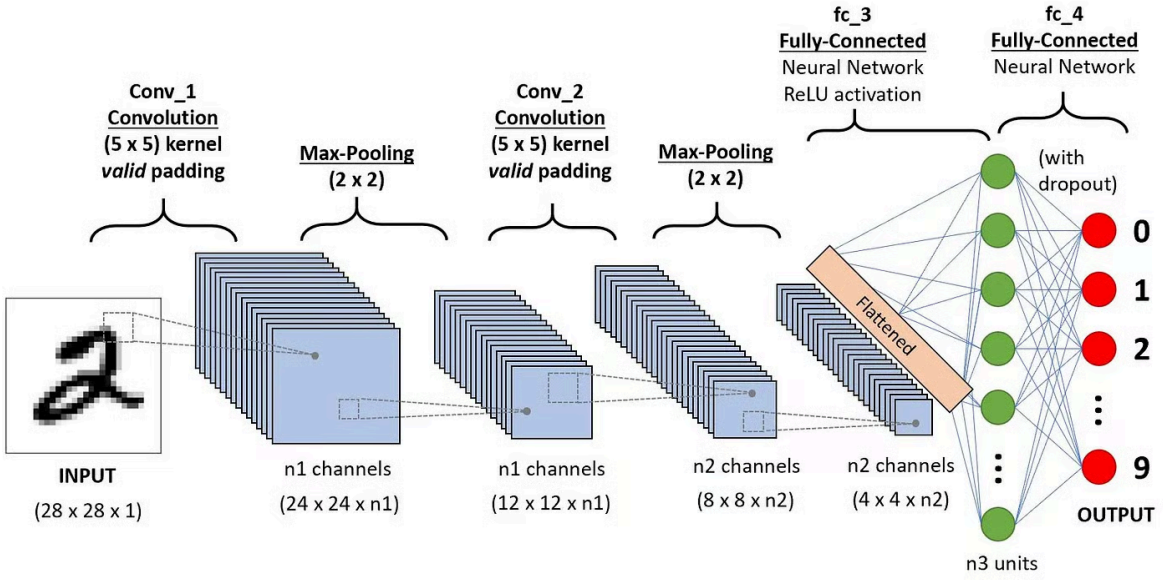
Convolutional neural networks were inspired by the layered architecture of the human visual cortex, and below are some key similarities and differences:

- **Hierarchical architecture:** Both CNNs and the visual cortex have a hierarchical structure, with simple features extracted in early layers and more complex features built up in deeper layers. This allows increasingly sophisticated representations of visual inputs.
- **Local connectivity:** Neurons in the visual cortex only connect to a local region of the input, not the entire visual field. Similarly, the neurons in a CNN layer are only connected to a local region of the input volume through the convolution operation. This local connectivity enables efficiency.
- **Translation invariance:** Visual cortex neurons can detect features regardless of their location in the visual field. Pooling layers in a CNN provide a degree of translation invariance by summarizing local features.

- **Multiple feature maps:** At each stage of visual processing, there are many different feature maps extracted. CNNs mimic this through multiple filter maps in each convolution layer.
- **Non-linearity:** Neurons in the visual cortex exhibit non-linear response properties. CNNs achieve non-linearity through activation functions like ReLU applied after each convolution.

The components of a CNN that help the CNNs mimic how the human brain operates to recognize patterns and features in images:

- Convolutional layers
- Rectified Linear Unit (ReLU for short)
- Pooling layers
- Fully connected layers



2.1.3.2 Discussion And Experimental Analysis

2.1.3.2.1 Convolutional Neural Network Configuration

Layer	Input	Output
First Convolutional: Conv2d(in_channels = 3, out_channels = 6, kernel_size = 5)	- tensor([N,3,32,32])	- Parameters: 6*3*5*5 + 6 = 456 - tensor([N,6,28,28])
First Max Pooling:	- tensor([N,6,28,28])	- tensor([N,6,14,14])

MaxPool2d(kernel_size = 2, stride = 2)		
Second Convolutional: Conv2d(in_channels = 6, out_channels = 16, kernel_size = 5)	- tensor([N,6,14,14])	- Parameters: 16*6*5*5 + 16 = 2416 - tensor([N,16,10,10])
Second Max Pooling: MaxPool2d(kernel_size = 2, stride = 2)	- tensor([N,16,10,10])	- tensor([N,16,5,5])
First Fully Connected: Linear(16 * 5 * 5, 120)	- Vector[x1, x2, …, x400] - 400 features	- Parameters: 400*120 + 120 = 48120 - 120 neurons
Second Fully Connected: Linear(120, 84)	- Vector[x1, x2, …, x120] - 120 neurons	- Parameters: 120*84 + 84 = 10164 - 84 neurons
Third Fully Connected: Linear(84, 10)	- Vector[x1, x2, …, x10] - 84 neurons	- Parameters: 84*10 + 10 = 850 - 10 neurons ~= 10 classes

2.1.3.2.2 Analysis

The table of training progress with the different **Learning Rate** and **Batch Size**
The number of **epoch** is **10**

Learning Rate	Batch Size	Train Loss	Test Loss	Train Accuracy	Test Accuracy
0.001	128	1.2452			

The Confusion Matrix of model with **Learning Rate** = , and **Batch Size** =

2.2 E2

2.3 E3

3. References

Asad, Muhammad. “Introduction to Softmax Classifier in PyTorch -

MachineLearningMastery.com.” *Machine Learning Mastery*,

https://machinelearningmastery.com/introduction-to-softmax-classifier-in-pytorch/.

Accessed 15 October 2026.